

AI AGENTS

Chapter 1: Introduction to AI Agents

By the end you'll be able to tell an agent from a workflow, name the three parts every agent has, and design a tool schema that keeps one in bounds.

MSBA AI Workshop · Session 1 · Sep 18, 2026 · 1:00–3:00 PM · 3039 BIF

Nine Fridays, four parts, one question at the end

Part I · Agents Sessions 1–3	1 Introduction to AI Agents	2 Prompt and Context Engineering	3 Memory, Retrieval and RAG
Part II · Machine learning Sessions 4–5	4 ML Foundations I	5 ML Foundations II	
Part III · Systems and decisions Sessions 6–7	6 Agent Systems	7 When to Use ML, Agents, or Neither	
Part IV · Finance Sessions 8–9	8 Financial Markets and AI as an Investment Theme	9 AI for Financial and Investment Research	

Everything before session 7 is a tool. Session 7 is when you decide which one to reach for.

You are here: session 1, the first of the three agent sessions.

DATES AND ROOMS

Every Friday afternoon, with two exceptions to note now

Part	Date	Time	Room	Topic
Part I · Agents	Sep 18	1:00 – 3:00 PM	3039 BIF	1. Introduction to AI Agents
Part I · Agents	Sep 25	1:30 – 3:30 PM	3007 BIF	2. Prompt and Context Engineering
Part I · Agents	Oct 2	1:30 – 3:30 PM	2043 BIF	3. Memory, Retrieval and RAG
Part II · Machine learning	Oct 16	1:30 – 3:30 PM	3007 BIF	4. ML Foundations I
Part II · Machine learning	Oct 23	1:30 – 3:30 PM	3007 BIF	5. ML Foundations II
Part III · Systems	Oct 30	1:30 – 3:30 PM	3007 BIF	6. Agent Systems
Part III · Systems	Nov 6	1:30 – 3:30 PM	3007 BIF	7. When to Use ML, Agents, or Neither
Part IV · Finance	Nov 13	1:30 – 3:30 PM	3007 BIF	8. Financial Markets and AI as an Investment Theme
Part IV · Finance	Dec 4	1:30 – 3:30 PM	3007 BIF	9. AI for Financial and Investment Research

Today starts at 1:00 in 3039 BIF; every later session is 1:30 in 3007 BIF, except Oct 2 in 2043 BIF. No session on Oct 9, Nov 20 or Nov 27.

WHERE EVERY CHAPTER IS SET

You work at Champaign Capital Research, and it has a problem

An independent equity research firm in Champaign, with a second office in Chicago. It sells written research on companies it covers — and everything it sells is made by hand.

38

people, including two
compliance officers

120

institutional clients
on annual subscriptions

~40

research notes
published a month

3

sectors: industrials, ag
equipment, semiconductors

A firm small enough that one person's afternoon is a real cost — and large enough to matter.

Fictional firm, real mechanics — every workflow you will automate is one a research firm actually runs.

WHO YOU WORK WITH

Five colleagues, and one empty desk that is yours

Priya Natarajan

Senior analyst, industrials — covers Deere and Caterpillar

Marcus Bell

Associate analyst, supports Priya on industrials

Jordan Lee

Analyst, semiconductors

Elena Ruiz

Head of compliance — clears about 30 staff trades a week

Tom Okafor

Data engineer — vendor feeds and model pipelines

You

Quantitative analyst on the investment team — that seat is yours

Three of them turn up in today's test cases — by name, with a trade to clear.

Nine sessions, one firm, a new problem on your desk each week.

You are the quantitative analyst on the investment team

You

Quantitative analyst

On the investment team, working next to the analysts and Tom's data team. Hired to automate work the firm still does by hand.

- 01 Take a workflow the firm runs by hand and judge what it actually needs.
- 02 Build it — agent, model, or plain code — and keep it inside guardrails.
- 03 Defend the choice to the people whose work it changes.

Nine weeks, nine problems on your desk. Today: the compliance queue.

Every exercise this term is written to you in that seat.

OPEN THIS NOW

The textbook is the lab — scan it before we start



Scan to open

`dl-msba-workshop.azurewebsites.net`

- Chapter 1 in full, with the agent loop running live in the page.
- The Colab notebook for today's exercises.
- Sign in with your Illinois account — the site holds the API key.

Every session has a chapter here; the slides are the tour, the site is where you build.

Today: where we're going

01 What an agent is

And when it's the wrong tool

02 Model, tools, loop

Three parts, three failure modes

03 The loop itself

Four lines, and what breaks them

04 Designing the tool API

Schemas beat prompts

05 Pre-clearance, live

Champaign Capital's compliance desk

01



What an agent is

The word is doing a lot of work in the market right now. We need a definition precise enough to build against.

THE PROBLEM ON THE DESK

~30



Trade pre-clearance requests reach Champaign Capital's compliance desk each week — handled by hand, over email.

Elena Ruiz clears every one of them by hand, and no two reviewers answer quite the same way.

Manual review is slow, but the real cost is inconsistency

- Same facts, different reviewer, different answer.
- The reasoning lives in an inbox, not in a record.
- Nothing links a decision back to the handbook clause behind it.

Hold onto these three — they are what we will judge the agent against.

DEFINITION

AI agent

A system where a language model decides its own next step: it asks for a tool, reads the result, and decides again — until it has an answer or runs out of budget.

WHY IT MATTERS

The model chooses the steps. That is the whole difference, and it is also where the risk lives.

Who picks the next step is the only question that matters

Shape	Who decides	Example
Single call	Nobody — one step	Summarize a document
Workflow	Your code, fixed order	Extract vendor, classify, draft email
Agent	The model, step by step	Clear a trade against the handbook

A workflow is a recipe. An agent is a cook.

Four questions decide whether you need an agent

- Steps known in advance? Use a workflow.
- Steps depend on what you find? Consider an agent.
- What does a mistake cost? Recommend before acting.
- Worth the extra calls, latency, and tokens?

Workflow by default; agent only when the recipe can't be written.

Three calls per trade is cheap; three calls per million rows is not

Good fit

Low volume, high judgment per item.

Steps vary with what's discovered.

A human signs off at the end.

Bad fit

High volume, identical every time.

Steps are already written down.

The action is irreversible.

Pre-clearance sits squarely in the left column — that is why we start here.

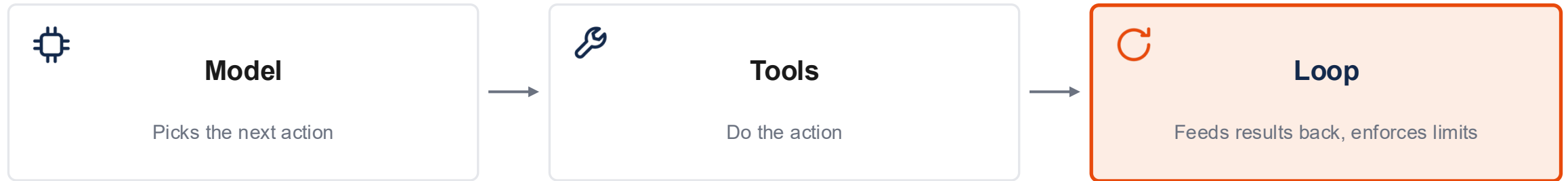
02



Model, tools, loop

Every agent is these three parts. When one misbehaves, knowing which one saves you an afternoon.

The model decides, the tools reach, the loop keeps it honest



Notice, act, check, repeat — the same habit a good compliance officer already has.

The model has no memory — it only sees what you hand it

- Reads the conversation so far, returns a tool call or an answer.
- Nothing carries over between calls.
- Anything it needs must be in the messages.

Wrong answer with the right tools? Look at the model or the prompt.

A tool is a typed contract, not a suggestion

- A name the model can call.
- A description of when to use it.
- A schema listing every allowed argument and its type.

Tool called with nonsense arguments? Look at the schema, not the prompt.

The loop is where your guardrails actually live

- Runs the tool and appends the result to the messages.
- Enforces a step budget and a timeout.
- Writes the log, and holds the approval gate.

A run that never ends? That's the loop, every time.

Read the symptom, and it tells you which part to fix

Symptom	Blame	Fix
Right tools, wrong answer	Model / prompt	Clearer instructions or context
Invalid arguments	Tool schema	Tighten types, enums, required fields
Never finishes	Loop	Step budget and timeout

This table is worth more to you in week three than any prompt-engineering tip.

03



The loop itself

Four lines of logic. Everything a framework adds sits on top of exactly this.

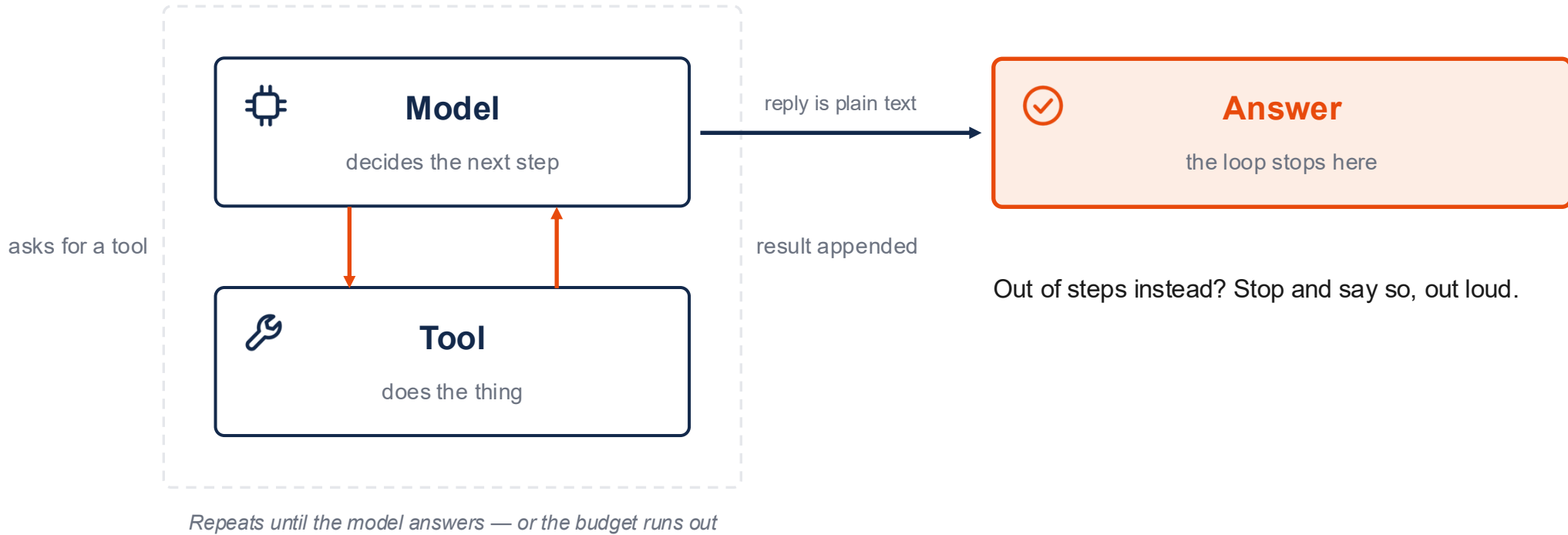
The agent loop, in four lines of plain English

- 1 Send the conversation so far to the model.
- 2 Reply is a tool call? Run it, append the result, go back to step 1.
- 3 Reply is plain text? Stop — that's the answer.
- 4 Out of steps? Stop and say so, out loud.

No hidden machinery. At any moment the conversation is readable end to end.

THE CYCLE

Two boxes and one exit — that is the entire architecture



Every agent framework you will meet is this picture with more logging.

The message list is the model's entire world, and it grows

Call 1

user
the question

Call 2

user
the question

assistant
get_trade_request

tool
held 12 days

Call 3

user
the question

assistant
get_trade_request

tool
held 12 days

assistant
search_docs

tool
30-day minimum

Same model, same tools — the only thing that changes is how much it can see.

Each pass is better informed than the last — that is the whole trick.

Two ways a loop goes wrong, and one guard each

Runaway loop

Keeps calling tools, never answers.

Guard: a step budget — six for one question.

Stops loudly: “step budget exhausted.”

Bad call

Calls a tool with invalid arguments.

Guard: catch it, return the error as a result.

The model reads it and corrects itself.

Both guards fail loudly. A silent failure teaches nobody anything.

04



Designing the tool API

The schema is the only hard boundary you have. A prompt asks; a schema refuses.

DEFINITION

Tool schema

The typed contract for one tool: its name, when to use it, and exactly which arguments it will accept — with everything else rejected before it runs.

WHY IT MATTERS

A prompt is advice the model may ignore. A schema is a wall it cannot walk through.

RULE ONE

Use enums wherever a free-text field would do

- A ticker field that accepts strings invites “Deere,” “DE Corp,” “deere.com.”
- An enum of valid symbols forces a real choice.
- Invalid values are refused before any code runs.

Every free-text argument is a guess you agreed to accept.

Mark fields required, and close the schema behind them

- Required means required — even with one valid value today.
- Close the schema so invented arguments are rejected.
- An ignored extra argument is a silent wrong answer.

Tomorrow there will be two valid values, and you'll be glad it was explicit.

RULE FOUR

Saying when not to call a tool beats saying when to

- “Use only when the user asks about revenue, growth, or earnings.”
- Over-calling is the most common failure in a new agent.
- One negative sentence in the description usually fixes it.

Cheaper than a prompt rewrite, and it travels with the tool.

Here is what the four rules look like in one tool

```
"name": "get_revenue"

"description": "Revenue and growth only.
               Do not call for prices."

"ticker":  enum ["DE", "CAT", "NVDA"]

"period":  string, required

"additionalProperties": false
```

- Says when NOT to call it
- Enum — no free text to misread
- Required — forces a real choice
- Closed — invented args rejected

Four rules, one tool, no prompt required.

Read it as a contract: everything not named here is refused.

Three of these four calls never reach your code

Call the model makes	Result	Caught by
ticker: DE, period: Q2-2026	Runs	—
ticker: Deere	Rejected	Enum
... plus currency: EUR	Rejected	Closed schema
query: (empty)	Rejected	Minimum length

Four rules, four classes of mistake you never have to debug.

05



Pre-clearance, live

Back to the compliance desk — now with an agent that recommends, and a human who decides.

0



Tools that can actually clear a trade. The agent has a lookup and a handbook search — nothing that acts.

What the agent can do is set by the tool list, not by what the prompt asks it to avoid.

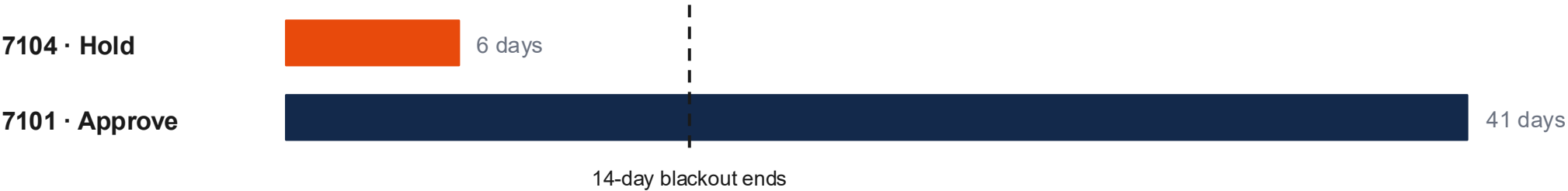
Four requests, four different reasons the answer differs

Request	Situation	Expected
7101 · Priya, buy 50 DE	Note published 41 days ago	Approve
7102 · Marcus, sell 200 CAT	Held only 12 days	Decline — 30-day minimum
7103 · Jordan, buy 30 NVDA	On the restricted list	Decline
7104 · Tom, sell 80 DE	Note published 6 days ago	Hold — 14-day blackout

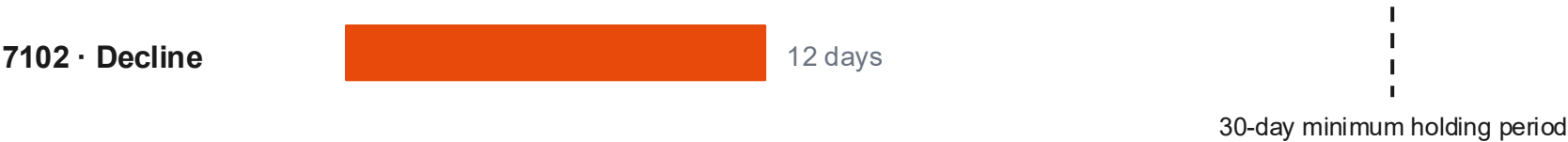
Same tools every time — only the discovered facts change the recommendation.

Three of the four answers turn on a date, not a judgment call

Clock 1 · days since we published on the name



Clock 2 · days the position has been held



7103 has no clock at all — NVDA is on the restricted list, so no number changes the answer.

Which is exactly why a handbook search beats asking the model what it remembers.

Tracing request 7102 through the loop, call by call

- 1 Model asks for the trade request. Result: Marcus has held CAT twelve days.
- 2 Model searches the handbook. Result: a thirty-day minimum holding period.
- 3 Model has enough. It returns text, and the loop stops.

Decline — twelve days against a thirty-day minimum [source: handbook, holding period].

The audit log is the deliverable, not a by-product

- Every recommendation carries the clause it rests on.
- Every step is logged: tool, arguments, result, elapsed time.
- Employees argue with the policy, not with the model.

Consistency, a written reason, and a trail — the three costs we opened with.

What to take away

- 01 An agent picks its own steps; a workflow follows yours. Default to the workflow.
- 02 Model, tools, loop — each owns a symptom, so each has its own fix.
- 03 Schemas prevent mistakes that prompts only discourage.
- 04 Recommending with a human gate is safer, and it is what earns the trust to do more.

Questions?

GO DEEPER

Run the loop yourself: dl-msba-workshop.azurewebsites.net — Chapter 1

Exercise: classify three tasks, then add a `get_price` tool with a closed schema

Anthropic, “Building effective agents” · OpenAI function-calling, strict schemas



The textbook, and the Colab